

Newton–Schulz Coefficients for the Muon Optimizer

1 Background

1.1 Newton–Schulz Orthogonalization

The Muon optimizer [1] uses Newton–Schulz (NS) iteration to orthogonalize gradient momentum matrices. For a weight matrix $W \in \mathbb{R}^{m \times n}$, the output of SGD-momentum, denoted $G_t \in \mathbb{R}^{m \times n}$, is fed into the iteration as a post-processing step:

$$X_0 = \frac{G_t}{\|G_t\|_F}, \quad X_{k+1} = aX_k + bX_kX_k^\top X_k + c(X_kX_k^\top)^2 X_k,$$

and the weights are updated as $W_{t+1} = W_t - \eta X_5$.

The iteration acts on the singular values of G_t through the scalar polynomial

$$\varphi(x) = ax + bx^3 + cx^5, \tag{1}$$

so that if $G_t/\|G_t\|_F = P\Sigma Q^\top$, then the NS output after k steps is $P\varphi^{(k)}(\Sigma)Q^\top$, where $\varphi^{(k)}$ denotes k -fold composition.

1.2 Existing Coefficient Sets

Two coefficient sets are provided as baselines:

	a	b	c	Source
Standard NS	1.500	−0.500	0.000	Classical degree-3 iteration
Jordan NS	3.445	−4.775	2.032	Numerical search [1]

Standard NS satisfies the exact constraints $\varphi(1) = 1$ and $\varphi'(1) = 0$, guaranteeing quadratic convergence near 1. However, $a = \varphi'(0) = 1.5$ provides weak pull from zero: after 5 iterations, a singular value of 10^{-3} reaches only 8×10^{-3} .

Jordan NS relaxes the fixed-point constraint ($\varphi(1) \approx 0.70$) in exchange for a larger $a = 3.44$. The same initial value reaches 0.47 in 5 iterations, yielding substantially faster convergence in practice, at the cost of increased step-to-step loss variance.

1.3 The Problem

Fixed coefficients cannot be optimal throughout training. In early stages, gradients are high-rank and noisy, favoring conservative a . In late training, gradients become low-rank, and aggressive a accelerates convergence. Moreover, $a = 3.44$ was found for a specific setting, and there is no guarantee it is optimal for other problems.

This exercise addresses two questions:

1. What is the maximal stable value of a , and what (b, c) support it?
2. Can a be self-tuned during training via a stability feedback signal, avoiding manual coefficient selection?

2 Stability Constraints

A candidate (a, b, c) triple must satisfy two conditions.

2.1 5-Iteration Band

After exactly $K = 5$ NS iterations, every initial singular value $x \in [\varepsilon, 1]$ where $\varepsilon = 10^{-3}$ must land in $[0.5, 1.5]$ (this range can be modified):

$$\forall x \in [\varepsilon, 1] : \quad 0.5 \leq \varphi^{(5)}(x) \leq 1.5. \quad (2)$$

The lower bound ensures effective orthogonalization; the upper bound prevents numerical overflow.

Standard NS fails this constraint: $\varphi^{(5)}(\varepsilon) \approx 0.008 < 0.5$. Jordan NS is on the borderline: $\varphi_{\min}^{(5)} \approx 0.47$.

2.2 Orbit Boundedness

No intermediate iterate may escape (this range can be modified too):

$$\forall x \in [\varepsilon, 1], \quad k = 1, \dots, 5 : \quad |\varphi^{(k)}(x)| \leq 2. \quad (3)$$

If $\varphi(x) > 2$, subsequent iterates may exceed the floating-point range, producing NaN quickly. This constraint eliminates coefficient sets (such as $a \approx 4.5$) that would otherwise appear admissible from the final-iterate band alone.

3 Tasks

Task 1: Stability Check

File: `muon/newton_schulz.py`

Function: `is_stable(a, b, c, epsilon=1e-3, n_pts=200)`

Given (a, b, c) , verify constraints (2)–(3). Sample n_pts log-spaced points in $[\varepsilon, 1]$, iterate the polynomial 5 times for each, and return True iff all points pass.

```
pytest tests/test_coeff_search.py::TestIsStable -v
```

Task 2: Random Coefficient Search

File: `muon/coeff_search.py`

Function: `search_coefficients(n_samples=50000, ...)`

Randomly sample (a, b, c) from specified ranges, evaluate each with `is_stable`, and return the triple with the largest a .

```
pytest tests/test_coeff_search.py::TestCoeffSearch -v
```

Task 3: Maximal Stable Coefficient

File: `muon/coeff_search.py`

Function: `find_fast_coeffs(n_samples=200000)`

Find the stable (a, b, c) with the maximum a . The result should exceed Jordan's $a = 3.44$.

```
pytest tests/test_coeff_search.py -k find_fast -v
```

Task 4: Coefficient Lookup Table

File: `muon/coeff_search.py`

Function: `build_coeff_table(a_values)`

For each target a in the input list, run a focused search to find the best stable (b, c) near that a . Return a dictionary $\{a_{\text{target}} \mapsto (a, b, c)\}$.

The adaptive optimizer's default table contains only two entries ($a = 1.5$ and $a = 3.44$). When the adaptive scheduler requests $a = 4.0$, nearest-key lookup returns $a = 3.44$, and the optimizer never uses aggressive coefficients. This task populates the table with intermediate entries so the adaptive method can function correctly.

Task 5: Adaptive Coefficient Scheduling

File: `muon/adaptive_optimizer.py`

Method: `AdaptiveMuon.step(loss_val)`

Implement loss-stability feedback:

Algorithm 1 Adaptive coefficient scheduling

```
1:  $a \leftarrow 3.0$ ;  $\bar{\ell}_t \leftarrow \text{None}$ ;  $\text{stable\_ct} \leftarrow 0$ 
2: for each step with loss  $\ell_t$  do
3:   if  $\bar{\ell}_t$  is None then
4:      $\bar{\ell}_t \leftarrow \ell_t$ 
5:   else
6:      $\bar{\ell}_t \leftarrow 0.99 \cdot \bar{\ell}_t + 0.01 \cdot \ell_t$ 
7:     if  $\ell_t > 3.0 \cdot \bar{\ell}_t$  then //NS iteration too aggressive
8:        $a \leftarrow \max(a - 0.2, 1.5)$ 
9:        $\text{stable\_ct} \leftarrow 0$ 
10:    else
11:       $\text{stable\_ct} \leftarrow \text{stable\_ct} + 1$ 
12:      if  $\text{stable\_ct} > 100$  then //NS iteration too conservative
13:         $a \leftarrow \min(a + 0.05, 4.0)$ 
14:      end if
15:    end if
16:  end if
17:  Look up  $(a, b, c)$  from table; apply NS; update weights
18: end for
```

```
pytest tests/test_adaptive.py -v
```

Task 6: Experiment

Run `python run.py`. This trains an MLP on random oscillatory functions and compares standard NS, Jordan NS, the fastest coefficients found by your search, and your adaptive optimizer. Output: `adaptive_ns_comparison.png`.

4 Open Discussion

1. Standard NS satisfies $\varphi(1) = 1$ and $\varphi'(1) = 0$ exactly, yet $\varphi^{(5)}(\varepsilon) \approx 0.008$. What does this reveal about the tension between asymptotic optimality near $x = 1$ and practical convergence for $x \ll 1$?
2. Jordan’s coefficients violate $\varphi(1) = 1$ by about 0.30. How far can this constraint be relaxed before the iteration becomes unstable? What mechanism couples a to the allowable deviation in $\varphi(1)$?
3. The adaptive method start at $a = 3.0$ and climbs toward $a = 4.0$. Would initializing directly at $a = 4.0$ work? If not, why?
4. The spike metric counts epochs where loss exceeds $3\times$ the running minimum. For aggressive coefficients, this count can reach several hundred. Are these instabilities or artifacts of aggressive exploration? Propose a metric that distinguishes the two.
5. Could a be adapted per-layer rather than globally? What signal would inform per-layer adaptation?

References

- [1] K. Jordan. Muon: An optimizer for hidden layers. kellerjordan.github.io/posts/muon/, 2024.
- [2] N. Amsel, T. Persson, C. Musco, R. Gower. The Polar Express: optimal matrix sign methods. *arXiv:2505.16932*, 2025.
- [3] J. Liu, J. Su, et al. Muon is scalable for LLM training. *arXiv:2502.16982*, 2025.